LONDON METROPOLITAN UNIVERSITY

Islington college
(इस्लिङ्टन कलेज)

Module Code & Module Title

CU6051NI Artificial Intelligence

75% Individual Coursework

Submission: Final Submission

Academic Semester: Autumn Semester 2025

Credit: 15 credit semester long module

Student Name: Amiks Karki

London Met ID: 23049352

College ID: NP01CP4A230172

Assignment Due Date: 21/01/2026.

Assignment Submission Date: 21/01/2026

Submitted To: Mr. Binod Bhattarai

GitHub Link	*https://github.com/AmiksKarki/Stock-Price-Prediction*

I confirm that I understand my coursework needs to be submitted online via MST Classroom under the relevant module page before the deadline for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table of Contents

Table of Figures

Table of Tables

1 Title of the Project

Comparative Analysis of Deep Learning Architectures (LSTM, CNN, and Transformers) for NEPSE Stock Price Prediction.

2 Introduction

The concept of machine learning has become a critical aspect of the modern artificial intelligence when the computational systems can identify patterns in the collections of the data and make the predictions that are not directly related to a set of rules that are personally developed. Within the task of analysis, the methods find extensive use in classification, regression, clustering and forecasting of sequential data. This has been enabled by the escalating availability of big data sets and the surge in computing capability which has enabled machine learning models to disclose hidden patterns in complicated information that has made it extremely significant in numerous industries such as healthcare, finance, transportation, manufacturing and e-commerce (Hapuarachchi, 2025).

Financial markets are highly dynamic in terms of behavior with high amount of uncertainty, noise and high non-linearities particularly in stock exchanges. It was extremely hard to predict, since the dynamics of the stock prices are predetermined but the interplay of the economic indicators, the events in the political sphere, the sentiment and investor psychology of the market, which is very difficult to forecast. Thus, price classification of stock has been one of the most challenging questions of machine learning in cases when there is a significant presence of very sensitive monetary time series information. Nonetheless, predictable and highly accurate predictive models can be of excellent importance to investors, financial institutions, and regulators since it allows competent decision-making (Seetharaman, 2021).

This study aims at developing a conceptual AI predictive stock price model of NEPSE (Nepal Stock Exchange) based on three common deep learning architectures: Long Short-Term Memory (LSTM) networks (Wenjie Lu, 2020), 1D Convolutional Neural Networks (CNN), and Transformer architectures. These are three angles of sequence modelling, which can be used to compare the performance of forecasting.

NEPSE is a market that is emerging and there is less research conducted on data-driven forecasting. It presents a perfect problem area since computation of smart forecasting models may lead to better decision-making of the Nepali financial system

2.1 Evolution from Milestone 1: Classification to Regression

Following the presentation of Milestone 1, and feedbacks on the same as provided by my tutor Mr. Binod Bhattarai, an in-depth re-consideration process was undertaken. It was found that the accuracy of classification was decent (around 55-60 percent), but the method did not make use of the abundant numerical data that financial time series data provides.

Based on this feedback, the methodology was radically changed to regression where the models forecast not where the direction moves but what the actual next-day closing price will be.

2.2 Key Deliverables:

- Full processing pipeline of prep and processing of NEPSE stock data.
- Execution of experimentation of 3 time-series prediction deep-learning structures.
- Severe performance appraisal and several metrics.
- Comparison of all the approaches with their advantages and drawbacks.
- The evidence-based recommendations.
- Full codebase including implementations reproducible and printed hyperparameters.

3 Background and literature review

3.1 NEPSE Stock Market Domain

Nepal Stock Exchange (NEPSE) is a vibrant financial market where share of various companies such as commercial banks, insurance companies, hydropower, telecommunications and manufacturing companies are traded.

NEPSE also includes a list of commercial banks, insurance companies, hydropower projects, telecom operators as well as manufacturing industry

Dataset Characteristics:

In this study, 17 commercial banks which were listed in NEPSE were selected since:

- The commercial banks possess the oldest and most complete trading histories.
- Attention to a single sector also minimizes confounding variables in an industry.
- Banking stocks, constitute one of the most heavily traded stocks on NEPSE.

Common historical price data: Historical prices are generally treated as a data set, describing the market in terms of a number of indicators, such as:

- Open price
- High price
- Low price
- Close price
- Percentage change
- Traded quantity
- Traded amount

It is an advantage that structured CSV data on NEPSE companies are available as a result of which it is possible to experiment with AI algorithms in prediction.

3.2 Time-Series Forecasting in AI

In time-series analysis, future data points are projected by learning temporal relationships from historical ordered data. Stock prices are a typical example, where the order of data

points is crucial. Traditional statistical models have limitations in capturing complex non-linear patterns, motivating the use of deep learning.

3.3 Deep Learning Approaches for Forecasting Financial Market Data

Deep learning models excel at feature extraction, non-linear pattern learning, and sequence modelling. The following architectures are widely used for financial forecasting:

3.3.1 Long Short-Term Memory (LSTM) Networks

Long Short-Term Memory networks extend the recurrent neural network framework by introducing memory cells and gating mechanisms that regulate information flow, allowing the model to retain relevant information over long temporal sequences.
Strengths:

- Good at learning temporal relationships
- Handles vanishing gradients
- Widely used for stock prediction

3.3.2 Convolutional Neural Networks (1D CNN) for Time-Series

Though commonly used in image processing, 1D CNNs can detect spatial/temporal patterns in sequential data.

Strengths:

- Captures local patterns
- Faster to train than LSTM
- Effective for short-term prediction windows

3.3.3 Transformer Models

Transformers use attention-based mechanisms rather than recurrent structures, which helps the model identify influential time steps and understand long-range patterns in sequential data.

Strengths:

- State-of-the-art performance in many time-series tasks
- Parallel training
- Works well with multi-series datasets

3.4 Review of Existing Literature

The models used in research in the field of global stock forecasting included LSTM, GRU, CNN, and Transformer-based models.

3.4.1 Study 1: Forecasting NEPSE Index Prices Using Deep Learning Techniques

- **Key Finding:** The LSTM model architecture has offered the greatest performance regarding the conventional evaluation measures, such as Root Mean Square Error (RMSE) and Mean Absolute Percentage Error (MAPE), in comparison to the GRU and CNN in processing the non-linear, complex, and volatile NEPSE time series data (Nawa Raj Pokhrel, 2022)

3.4.2 Study 2: Financial Market Prediction Employing Long Short-Term Memory Networks

- **Key Findings:** LSTM was extremely efficient in comparison with conventional ML (Random Forests, DNNs) because it was able to identify long-term correlations. Normalization (preprocessing, sequence formation) was important (Thomas Fischer, 2018).

3.4.3 Study 3: Comparative Analysis of RNN, LSTM, BiLSTM, GRU, and Transformer Models for Financial Forecasting

- **Key Findings:** Transformer demonstrated better accuracy and more effective convergence to all setting of RNN (LSTM, GRU), which proved the advantage of the self-attention mechanism (T.O. Kehinde, 2023).

3.4.4 Study 4: Stock Prediction Based on Transformer Model

- **Key Findings:** The combination of the Transformer and LASSO regularization

performed significantly better in the context of robustness and had low MAPE, which indicates that feature engineering is needed to the real NEPSE forecast (Ekanta Rai, 2025).

3.4.5 Study 5: A Deep Neural Network-Based Stock Trading System Incorporating Evolutionary Optimized Technical Analysis Parameters

- **Key Findings:** CNN was more effective in high-frequency trading (speed). Predictions by LSTM were more stable in longer investment horizons (Omer Berat Sezera, 2017).

These studies show:

- The sequential memory is well recalled by LSTM.
- CNN is effective in the extraction of patterns.
- Both due to attention mechanisms tend to be outperformed by transformers.
- In Nepal, there is a paucity of academic literature on computational stock prediction, which creates an opportunity of AI-based stock forecast to generate value..

3.5 Problem Statement

Prediction of stock prices in the emerging stock markets like NEPSE are so difficult to predict because the data is complicated, the markets are volatile and systematic comparative study using advanced deep learning architecture is not available. Moreover, minimal literature is done on the use of the state-of-the-art models like Transformers on NEPSE data and it is not known whether certain algorithms are more effective in this market environment.

In this paper, the primary issue fashioned is the following issue:

- What is the comparison of LSTM with CNN and Transformer model in their ability to predict next-day stock price of the companies listed on NEPSE?

With the solution to this issue, the study will result in evidence-based recommendations of selecting appropriate deep learning-based architectures in predicting stock prices in future markets, which will aid in better decision making.

3.6 Research Questions / Objectives

3.6.1 Research Questions

1. Which deep learning architecture (LSTM, CNN or Transformer) has the most accurate prediction accuracy of NEPSE Stock Price Forecasting?
2. How are the preprocessing steps such as data normalization, sequence generation and feature encoding affecting the performance of each model?
3. What are the computation efficiency differences between the LSTM, the CNN, and the Transformer model in terms of time of training and the number of resources needed?
4. Which are the most suitable evaluation metrics in predicting financial time series data in real-world scenarios?

3.6.2 Research Objectives

1. Develop a unified multi-company dataset by incorporating historical stock data from NEPSE-listed companies.
2. Design and implement preprocessing pipelines such as data cleaning, feature scaling and sequence generation suitable for deep learning models.
3. Develop three architectures for deep learning: LSTM, CNN and Transformer which are optimized for time series regression using the same input data to compare the three architectures fairly.
4. Train all the three models with the same hyperparameters, training protocols and validation strategies.
5. Evaluate model performance using several metrics and visualize comparison results.
6. Issue evidence-based suggestions for algorithm choice and provide evidence for selecting sensitive, efficient, and implementation feasible for algorithms.

4 Solution

4.1 Proposed Solution Overview

This study, regression structure is applied in a structured way in which it is supervised to estimate the forthcoming closing price on a day-by-day basis using past 60 day sequence data. There are seven key stages of solution pipeline:

- Data Collection: Information on aggregate historical trading of 17 NEPSE commercial banks.
- Feature Engineering: Choose best features.
- Sequence Generating: Generate overlapping 60-day windows and targets.
- Temporal Splitting: Segregate data based on time in order to avoid information leakage in the future.
- Normalization: Direct normalization of features and targets.
- Model Training To train three architectures using the same protocols.
- Performance: Comparison of multiple regression metrics.

4.1.1 Proposed AI Algorithms

Algorithm	Architecture Type	Reason for Selection
LSTM	Recurrent Neural Network (Deep Learning)	LSTM Recurrent Neural Network (Deep Learning) The LSTM networks are intended to deal with moving data over time, and its efficacy to capture lasting time-based associations through memory cells and gating apparatuses. They meet the vanishing gradient challenge of standard RNNs as well as have already proved performance of great financial time-series forecasting by learning fundamental historical patterns of prices even after long durations (Thomas Fischer, 2018).

CNN (1D)	Convolutional Neural Network (Deep Learning)	CNN (1D) Convolutional Neural Network (Deep Learning) The CNNs are one-dimensional convolutions, and they are efficient to extract local time series information temporal patterns. They can be used to find fast growing and declining trends in the market and produce faster training and computation performance than recurrent models (Avinash, 2021).
Transformer	Attention-based Neural Network (Deep Learning)	Transformer Attention-based Neural Network (Deep Learning) Transformers are based on self-attention to systematically model both short-term and long-term dependencies, without the use of recurrence. Their parallel processing nature enhances training speed and more recent research indicates that they are better than traditional architecture in time-series predictions tasks (Rogerio Pereira dos Santos, 2025).

Table 1: Proposed Algorithms

4.1.2 Dataset Description

- Dataset Source: Historical stock price data collected from Nepal Stock Exchange (NEPSE) official records made publicly available via GitHub repository. The repository is continuously updated using automated web-scraping techniques to reflect daily market activity.
- Type: Time-series financial dataset containing daily stock trading information.
- Instances: Each instance represents one trading day for a listed stock. The dataset spans multiple years with new records appended on each trading day.
- Features: The dataset used in this study includes the following variables:
    - published_date: published_date: Date of each trading session.
    - open: The price of the stock when the trading day opens.
    - high: This is the highest price at which the trading session moved.
    - low The lowest price in the session.

- close: The market close price.
- per_change: the change of the closing price in percentage with respect to the previous trading day.
- quantity traded:Total number of shares traded.
- traded amount: Monetary amount of trading amount of shares.
- company id: Company identifier.

- Engineered Features:

  First, extensive technical indicators were developed using raw information in the field of trading. These indicators include trend, momentum, volatility, volume and price based indicators. The prediction target was determined by the next-day closing price.

  It was demonstrated that the predictive performance was not enhanced by the technical indicators through experimental means. The more the input features included, the greater the noise, overfitting and more time was needed to train and the prediction was always less accurate. A small representation of features produced the best performance in all the models thus the following features were finalized:

  - Input Features:
    - Historical Closing Price (60-day sequence)
    - Momentum (Percentage Change)
    - Learned vector representations for each of 17 banks(Company Embeddings)
  - Target Variable
    - Next-Day Closing Price
- Training Set: 80% temporal split (39,632 sequences, ~76.2%)
- Test Set: 20% temporal split (12,411 sequences, ~23.8%)
- Missing Values: The dataset contains occasional missing values which are handled during the preprocessing stage using appropriate techniques.

4.2 Pseudocode

4.2.1 Dataset Creation and Sequence Generation

```
BEGIN DataPreparation
  // Load and combine data
  Load 17 commercial bank CSV files
  Add company identifiers
  Merge into single dataset
  Sort by company and date

  // Feature engineering
  Calculate momentum as percentage change
  Fill missing values with zero

  // Create sequences per company
  FOR each company
    Create 60-day sliding windows
    Each sequence contains close price and momentum
    Target is next day closing price
  END FOR

  // Temporal train-test split
  Split data at 80% cutoff date
  Create training set (76.2% of sequences)
  Create test set (23.8% of sequences)

  // Feature scaling
```

```
    Fit MinMaxScaler on training targets
    Transform both training and test targets
    Fit StandardScaler on training features
    Transform both training and test features

    // Create data loaders
    Create PyTorch datasets with sequences
    Create batched data loaders (batch size 32)

    RETURN train_loader, test_loader, scalers
END
```

4.2.2 LSTM Model

BEGIN LSTMModel

// Build model architecture

Create company embedding layer (17 companies to size 4)

Add LSTM layer with 50 hidden units

Add fully connected layer combining LSTM and embedding

// Forward pass

Process sequence through LSTM

Extract last time step features

Get company-specific embedding

Concatenate LSTM output with embedding

Generate price prediction

// Configure training

Set optimizer to Adam with learning rate 0.001

Set loss function to MSE

Configure early stopping with patience 15

Create validation split (90/10)

// Train model

WHILE training not complete

Process batch of training data

Calculate prediction error

Update model weights

Check validation performance

```
        IF validation stops improving THEN
            Stop training
        END IF
    END WHILE

    // Return trained model
    Load best model checkpoint
    RETURN trained model
END
```

4.2.3 CNN Model

BEGIN CNNModel

// Build model architecture

Create company embedding layer (17 companies to size 4)

Add convolutional layers with kernel sizes 3

Add max pooling layers

Add fully connected layers with dropout

// Forward pass

Transpose input for convolution

Apply convolutional layers with ReLU

Apply pooling layers

Flatten CNN features

Get company-specific embedding

Concatenate CNN output with embedding

Generate price prediction through FC layers

// Configure training

Set optimizer to Adam with learning rate 0.001

Set loss function to MSE

Configure early stopping with patience 15

// Train model

WHILE training not complete

Process batch of training data

Calculate prediction error

```
    Update model weights
    Check validation performance
    IF validation stops improving THEN
        Stop training
    END IF
  END WHILE

  RETURN trained model
END
```

4.2.4 Transformer Model

```
BEGIN TransformerModel
  // Build model architecture
  Create company embedding layer (17 companies to size 4)
  Add input projection layer (2 features to 32 dimensions)
  Add transformer encoder (2 layers, 4 attention heads)
  Add fully connected output layers

  // Forward pass
  Project input features to higher dimensions
  Process through transformer encoder layers
  Apply global average pooling
  Get company-specific embedding
  Concatenate transformer output with embedding
  Generate price prediction

  // Configure training
  Set optimizer to Adam with learning rate 0.001
  Set loss function to MSE
  Configure early stopping with patience 15

  // Train model
  WHILE training not complete
     Process batch of training data
     Calculate prediction error
     Update model weights
     Check validation performance
     IF validation stops improving THEN
        Stop training
     END IF
  END WHILE
```

```
  RETURN trained model
END
```

4.2.5 Model Evaluation

```
BEGIN ModelEvaluation
  // Evaluate each model
  FOR each model (LSTM, CNN, Transformer)
    Generate predictions on test set
    Inverse transform to original price scale
    Calculate RMSE (root mean squared error)
    Calculate MAE (mean absolute error)
    Calculate R-squared score
    Calculate MAPE (mean absolute percentage error)
  END FOR

  // Compare results
  Create results table with all metrics
  Identify best model by lowest RMSE

  // Visualize performance
  Plot predictions vs actual prices
  Plot metric comparison charts
  Plot training history curves

  RETURN comparison results and best model
END
```

4.3 Diagrammatic Representation

4.3.1 Overall System Flowchart

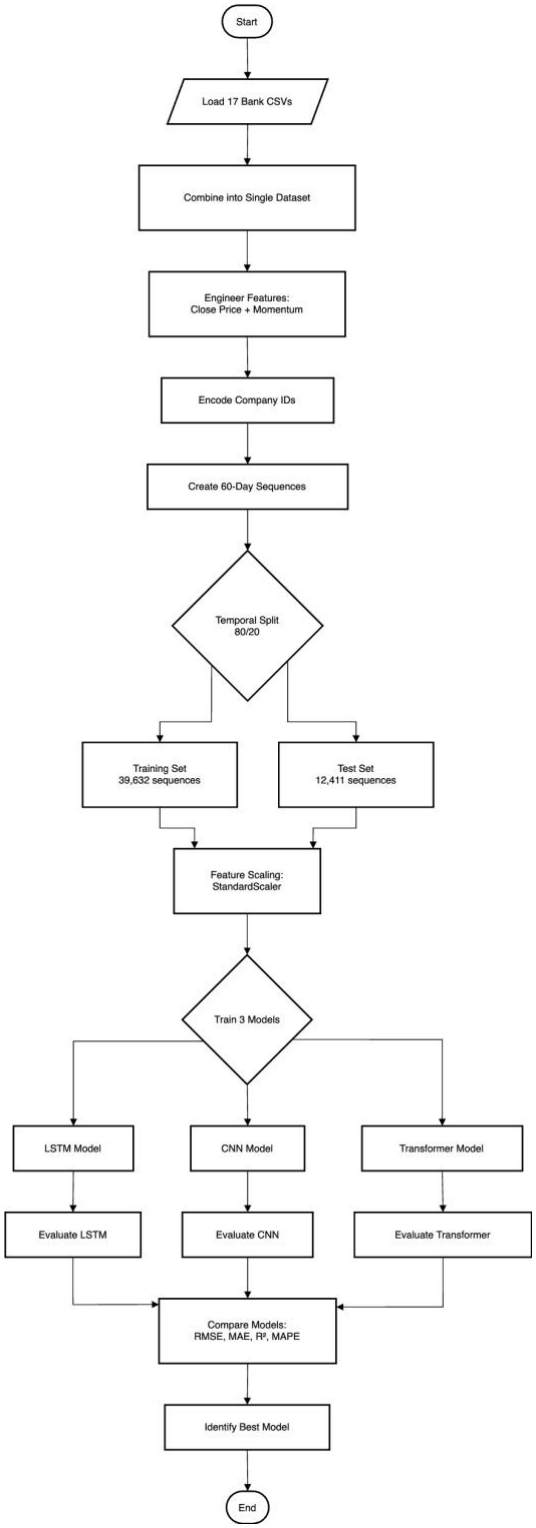


Figure 1: Overall System Flowchart

4.3.2 Data preprocessing flowchart

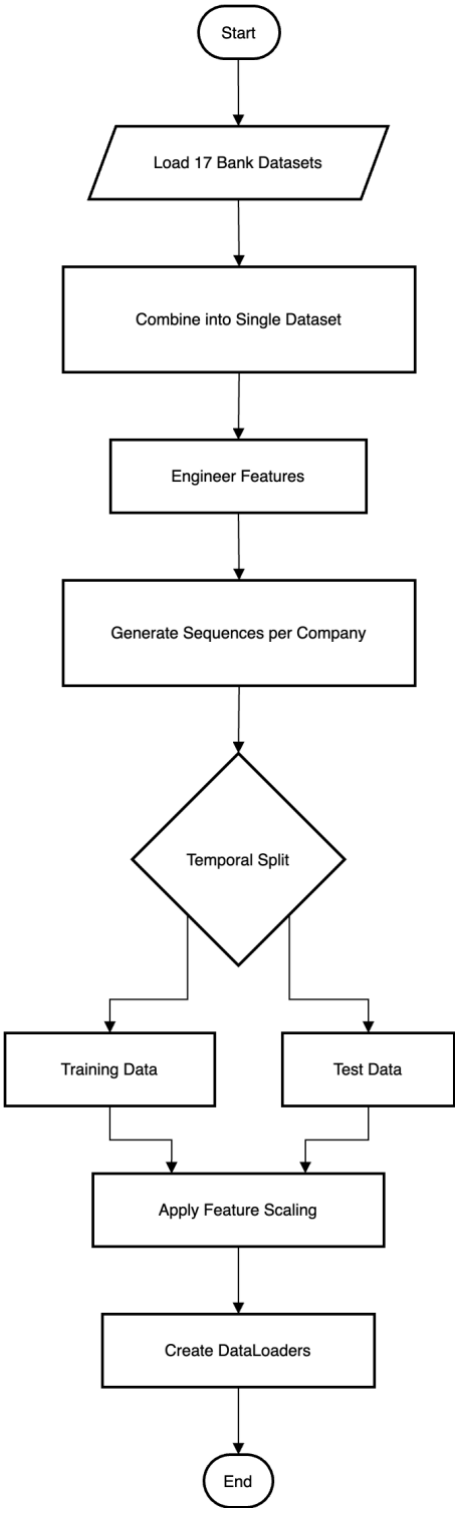


Figure 2: Data Preprocessing

4.3.3 LSTM Model Implementation Flowchart

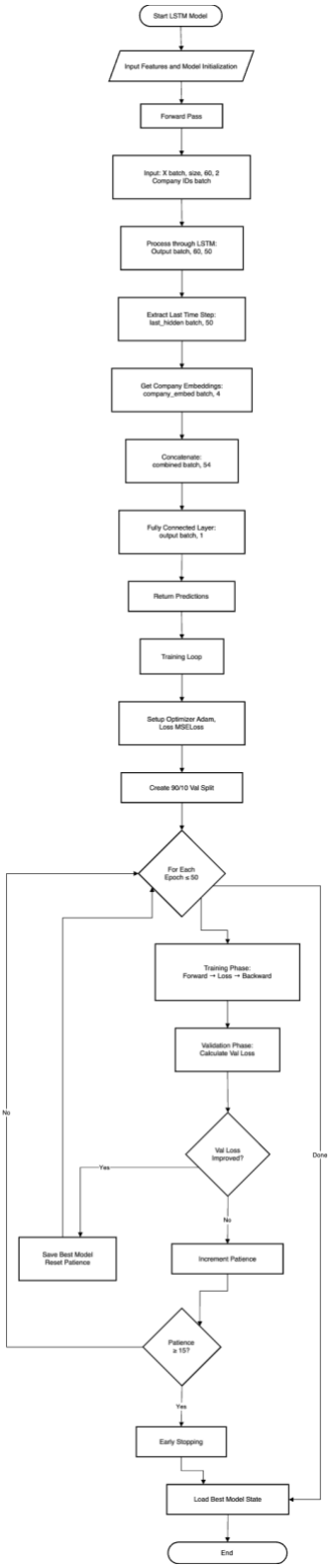


Figure 3: LSTM Implementation

4.3.4 CNN Model Implementation Flowchart

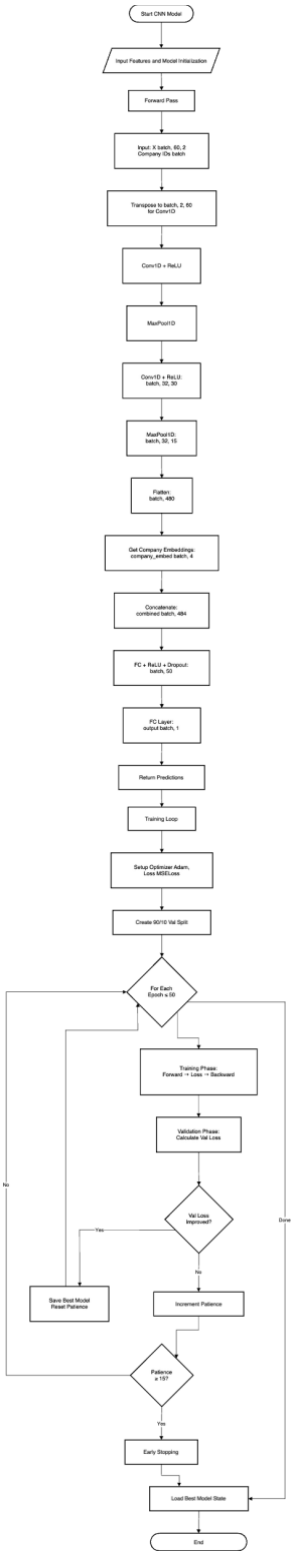


Figure 4: CNN Implementation

4.3.5 Transformer Model Implementation Flowchart

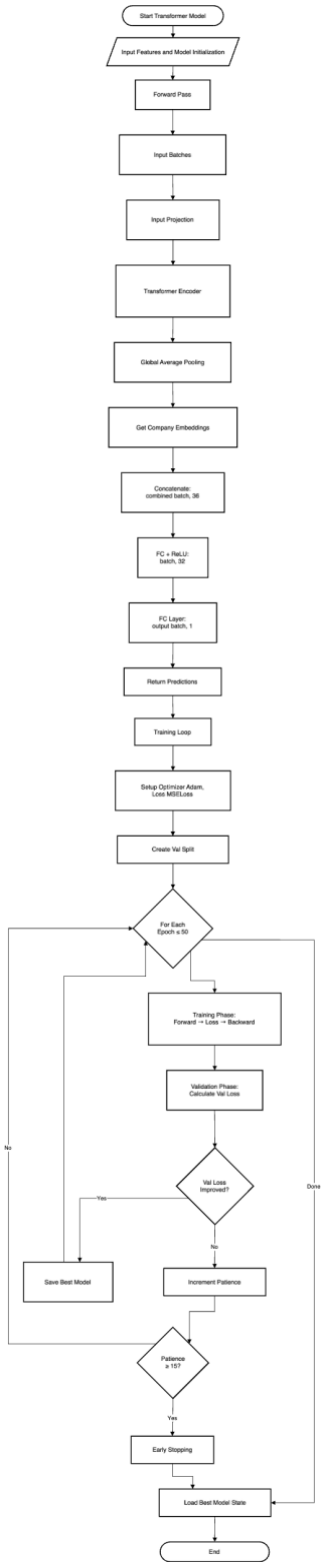


Figure 5: Transformer Implementation

4.4 Explanation of the development process

4.4.1 Development Workflow

4.4.1.1 Setup and Exploration

- Configured Google Colab environment
- Loaded and inspected 17 commercial bank datasets
- Analyzed data quality, completeness, and distribution
- Created initial exploratory visualizations
- Set random seeds (42) for reproducibility

4.4.1.2 Data Preprocessing

4.4.1.2.1 Dataset Combination:

- Merged 17 individual bank datasets into unified DataFrame
- Added company_id column to maintain bank identity
- Sorted chronologically by (company_id, published_date)
- Final combined dataset: 53,063 rows × 10 columns

4.4.1.2.2 Feature Engineering:

- Feature 1 (Close Price): Direct closing price values
- Feature 2 (Momentum): Percentage change

4.4.1.2.3 Company Encoding:

- Created company-to-ID mapping: {ADBL: 0, CZBIL: 1, ..., SCB: 16}
- Total unique companies: 17
- Encoded as integers for embedding layer compatibility

4.4.1.2.4 Sequence Generation:

- Implemented 60-day sliding window approach
- Generated sequences per company to prevent cross-contamination
- Each sequence (60 days)
- Target: Next-day closing price (day 61)

4.4.1.2.5 Temporal Split:

- Calculated cutoff date: 80% of (max_date - min_date)
- Training set: 39,632 sequences (76.2%) before cutoff
- Test set: 12,411 sequences (23.8%) after cutoff
- This ensures no future information leaks into training

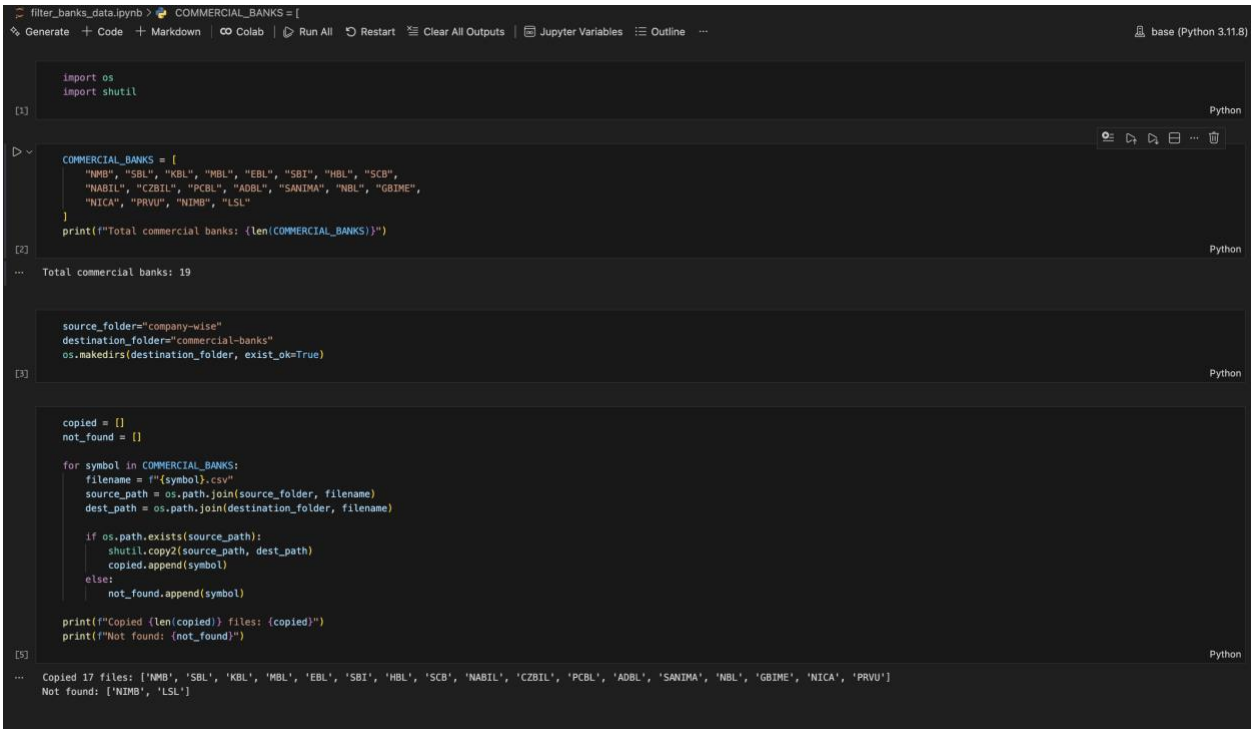
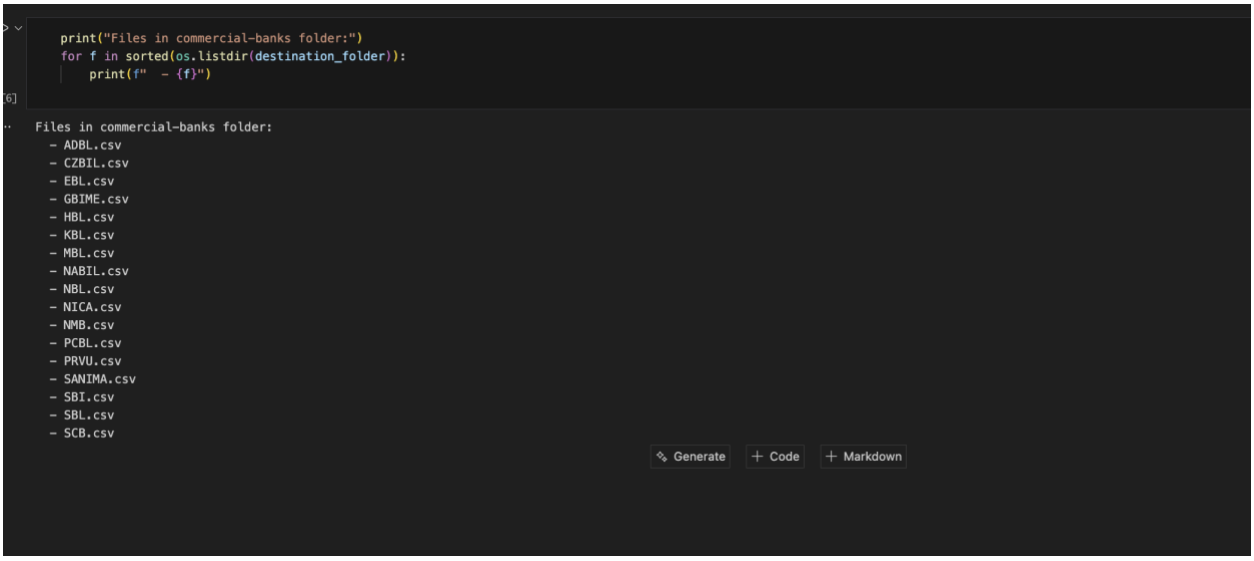


Figure 6: Filtering commercial banks data

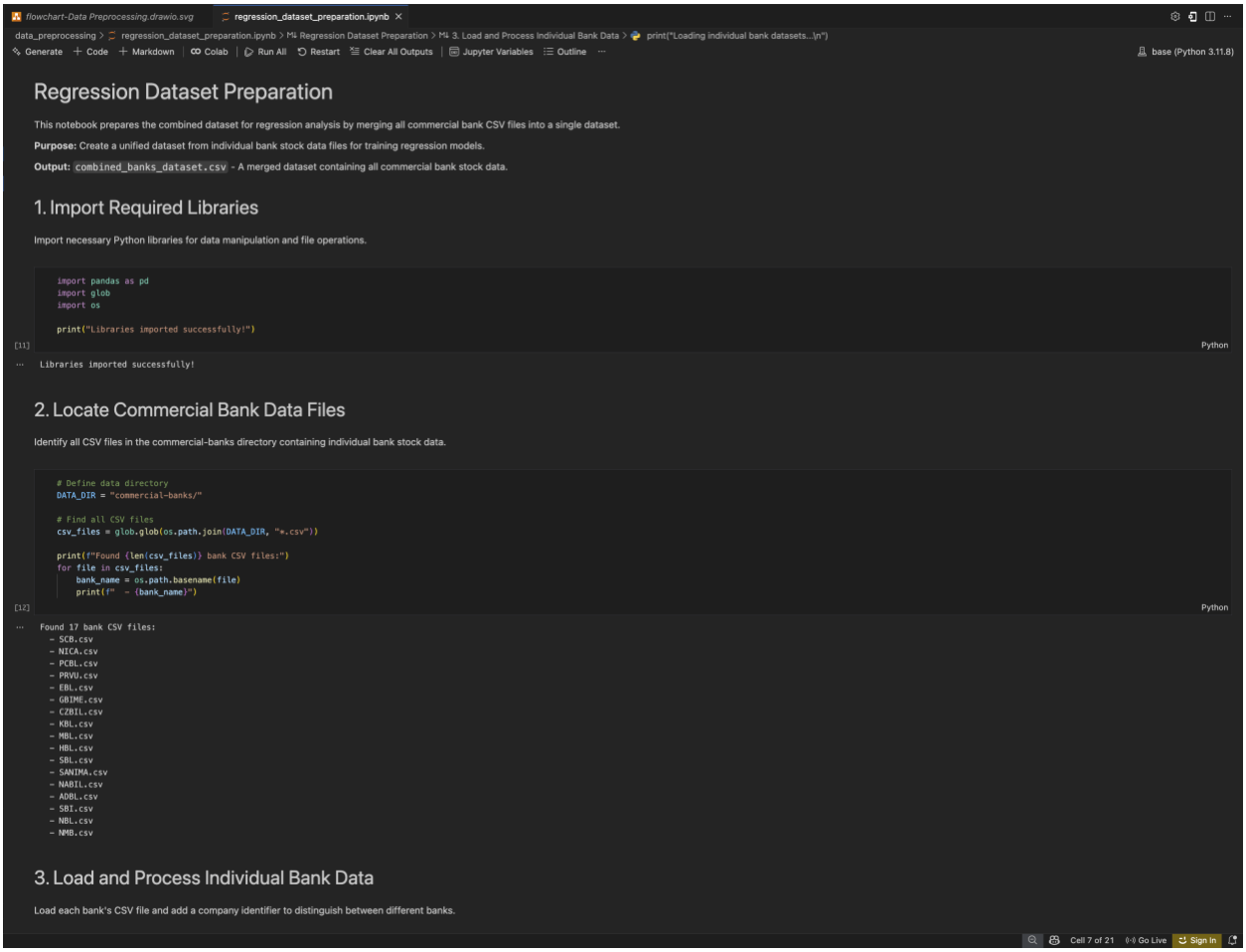


```
    print("Files in commercial-banks folder:")
    for f in sorted(os.listdir(destination_folder)):
        print(f"  - {f}")
```

```
Files in commercial-banks folder:
  - ADBL.csv
  - CZBIL.csv
  - EBL.csv
  - GBIME.csv
  - HBL.csv
  - KBL.csv
  - MBL.csv
  - NABIL.csv
  - NBL.csv
  - NICA.csv
  - NMB.csv
  - PCBL.csv
  - PRVU.csv
  - SANIMA.csv
  - SBI.csv
  - SBL.csv
  - SCB.csv
```

Generate + Code + Markdown

Figure 7: Saving the commercial banks data in the respective folder



flowchart-Data Preprocessing.drawio.svg | regression_dataset_preparation.ipynb

data_preprocessing > regression_dataset_preparation.ipynb > M↓ Regression Dataset Preparation > M↓ 3. Load and Process Individual Bank Data > print("Loading individual bank datasets...\n")

Generate + Code + Markdown | Colab | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline | base (Python 3.11.8)

Regression Dataset Preparation

This notebook prepares the combined dataset for regression analysis by merging all commercial bank CSV files into a single dataset.

Purpose: Create a unified dataset from individual bank stock data files for training regression models.

Output: `combined_banks_dataset.csv` - A merged dataset containing all commercial bank stock data.

1. Import Required Libraries

Import necessary Python libraries for data manipulation and file operations.

```
import pandas as pd
import glob
import os

print("Libraries imported successfully!")
```

[11] Python

```
Libraries imported successfully!
```

2. Locate Commercial Bank Data Files

Identify all CSV files in the commercial-banks directory containing individual bank stock data.

```
# Define data directory
DATA_DIR = "commercial-banks/"

# Find all CSV files
csv_files = glob.glob(os.path.join(DATA_DIR, "*.csv"))

print(f"Found {len(csv_files)} bank CSV files:")
for file in csv_files:
    bank_name = os.path.basename(file)
    print(f"  - {bank_name}")
```

[12] Python

```
Found 17 bank CSV files:
  - SCB.csv
  - NICA.csv
  - PCBL.csv
  - PRVU.csv
  - EBL.csv
  - GBIME.csv
  - CZBIL.csv
  - KBL.csv
  - MBL.csv
  - HBL.csv
  - SBL.csv
  - SANIMA.csv
  - NABIL.csv
  - ADBL.csv
  - SBI.csv
  - NBL.csv
  - NMB.csv
```

3. Load and Process Individual Bank Data

Load each bank's CSV file and add a company identifier to distinguish between different banks.

Cell 7 of 21 | Go Live | Sign In

Figure 8: Locating commercial banks data files

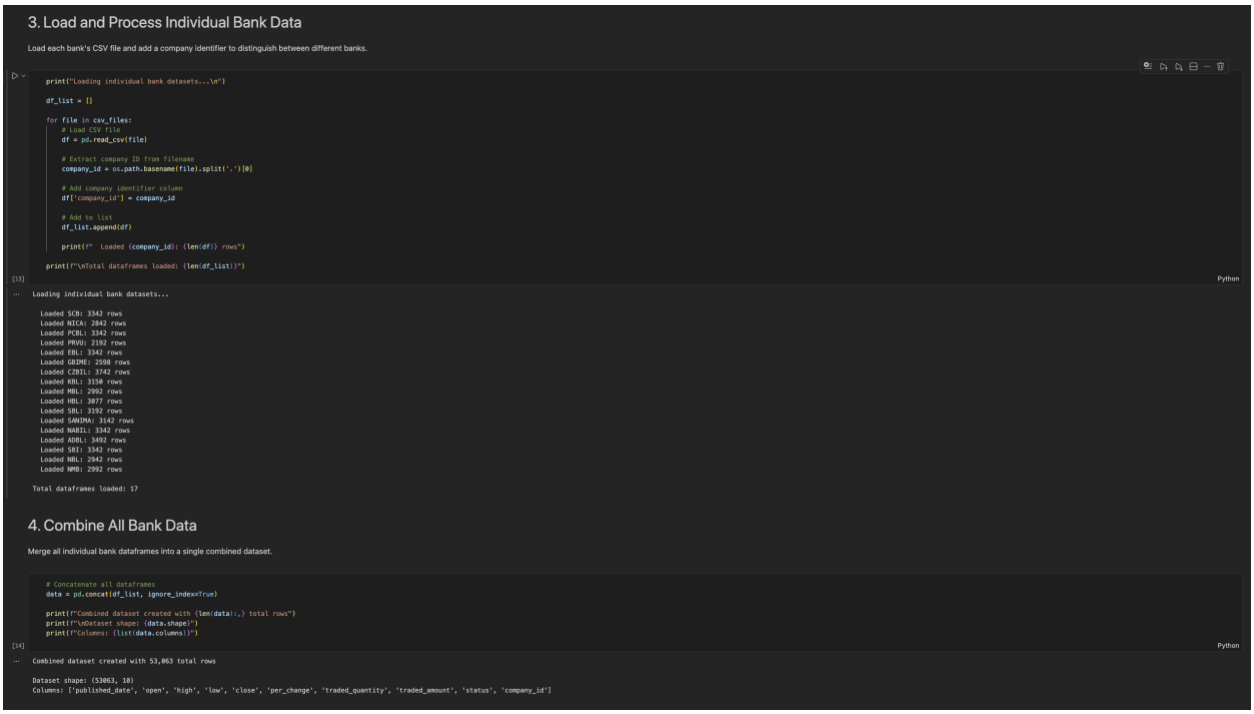


Figure 9: Loading, Processing and combining the banks data into single one

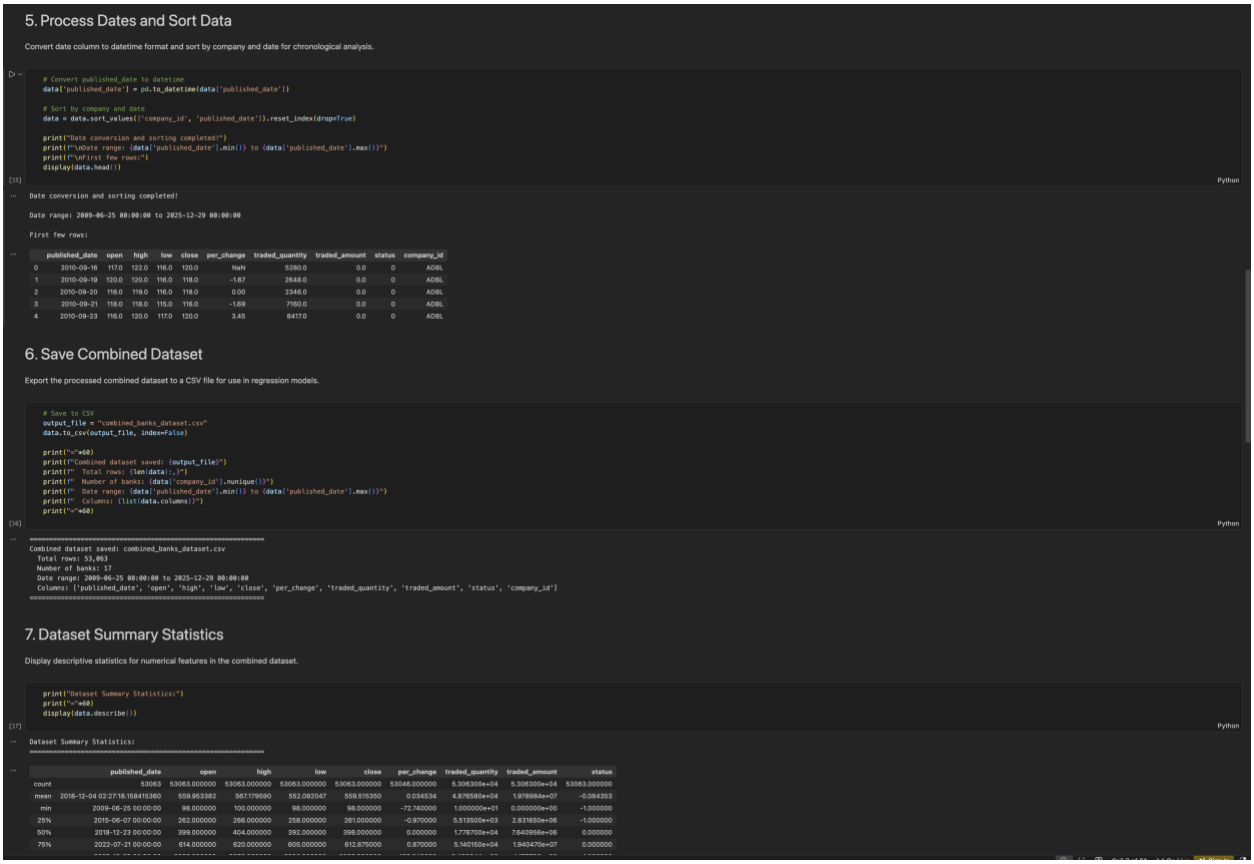


Figure 10: Saving the combined data into single csv

4.4.1.3 Dataset Visualization

To achieve an answer to the question of how NEPSE stock price models were to be trained, the various exploratory visualizations are drawn of the data. The data were analyzed with the help of multiple visualizations in order to discover the distribution of the data and potential anomalies.

The following visualizations were included:

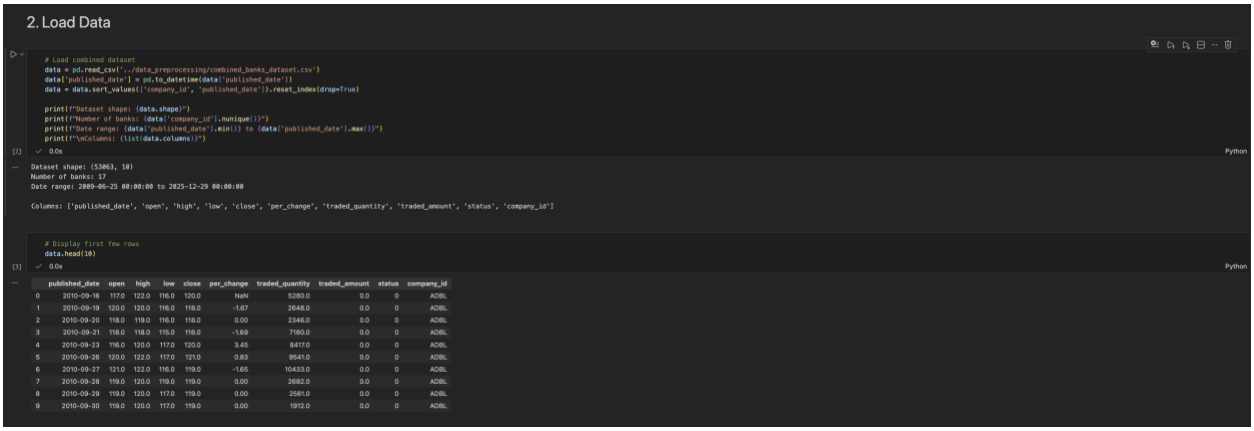


Figure 11: Data Loading

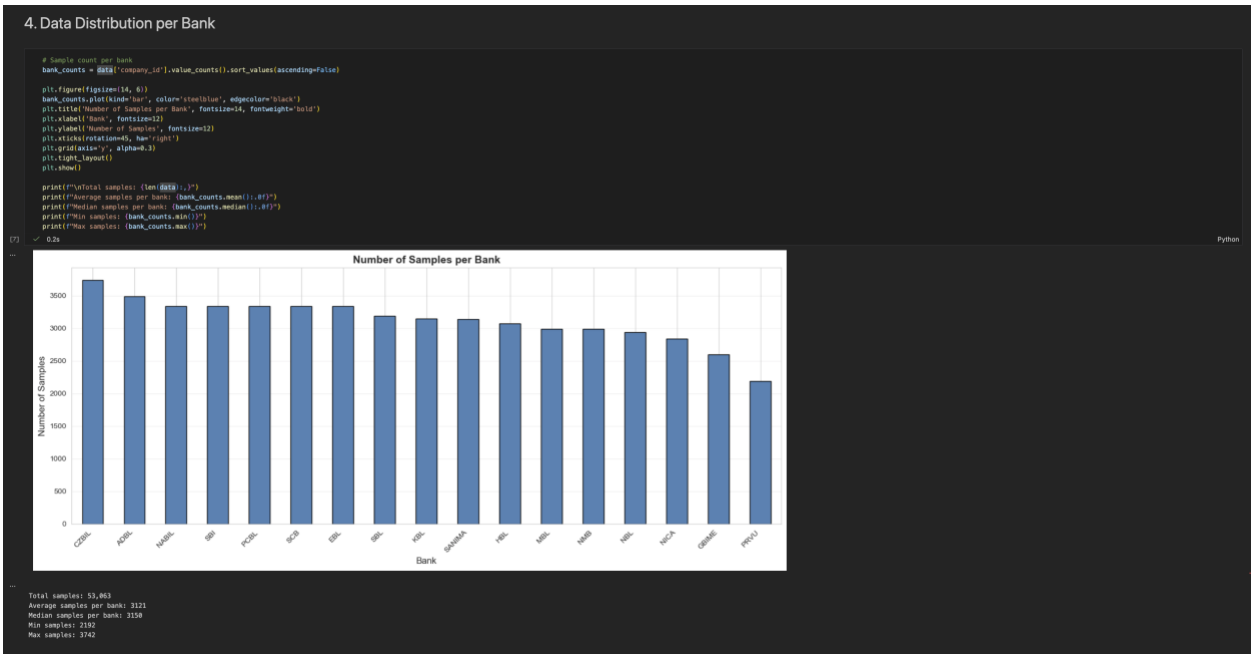


Figure 12: Data Distribution Visualization

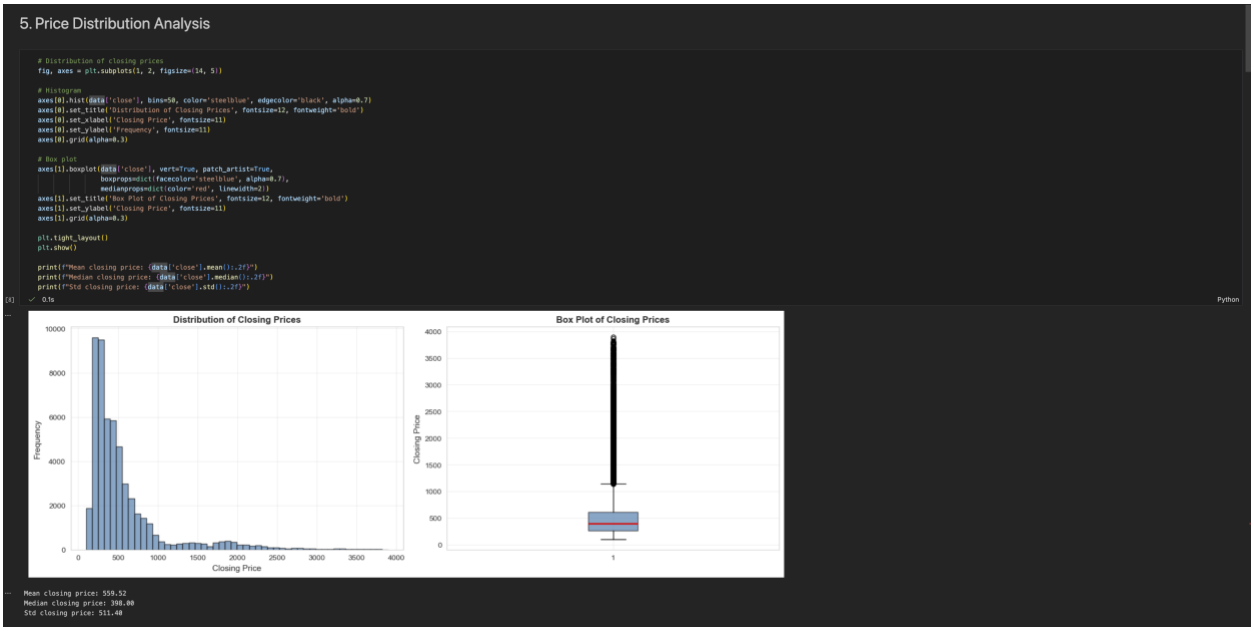


Figure 13: Price Distribution

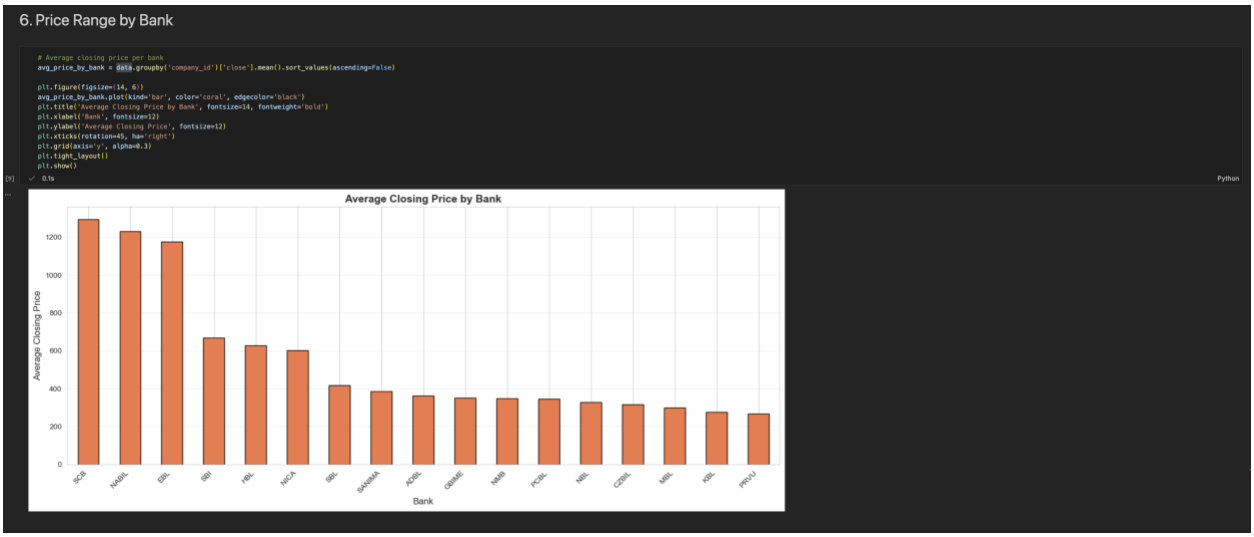


Figure 14: Visualization of closing price range of all banks

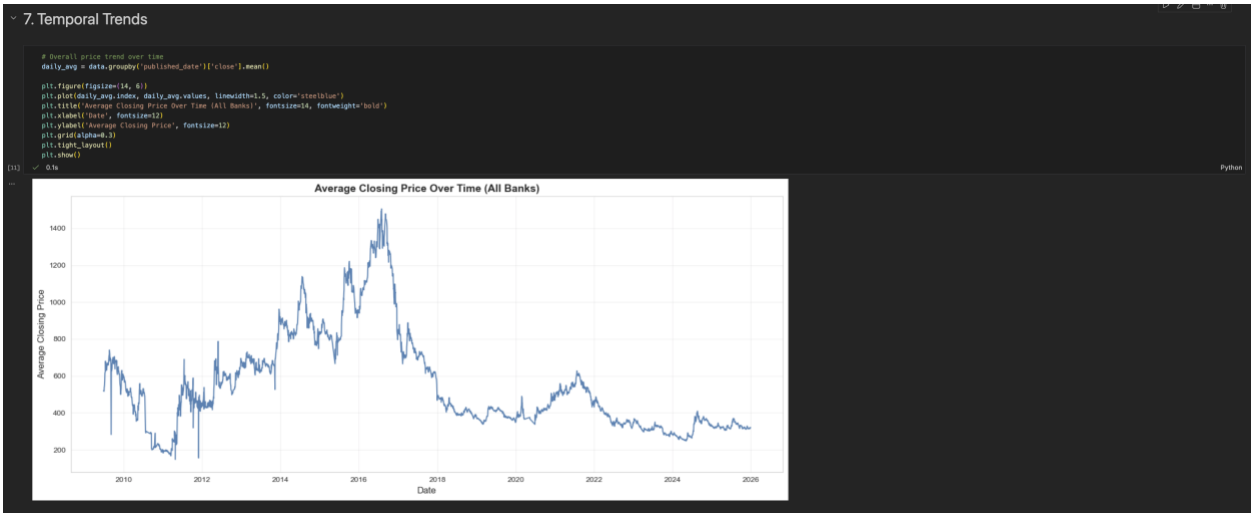


Figure 15: Average Closing Price Visualization over time

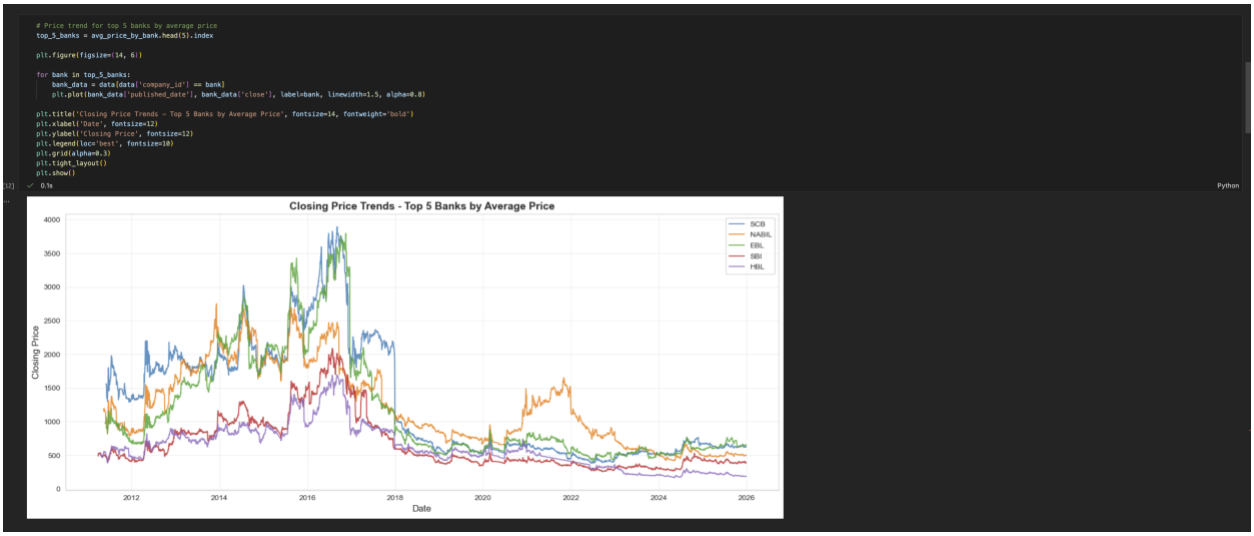


Figure 16: Closing Price Trends of 5 banks

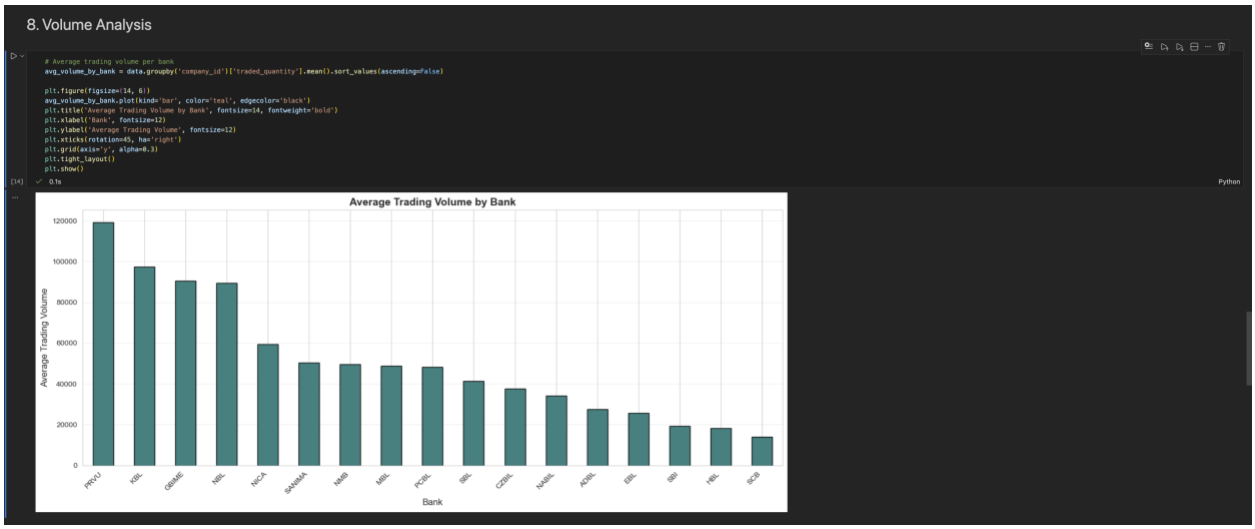


Figure 17: Avg Trading Volume

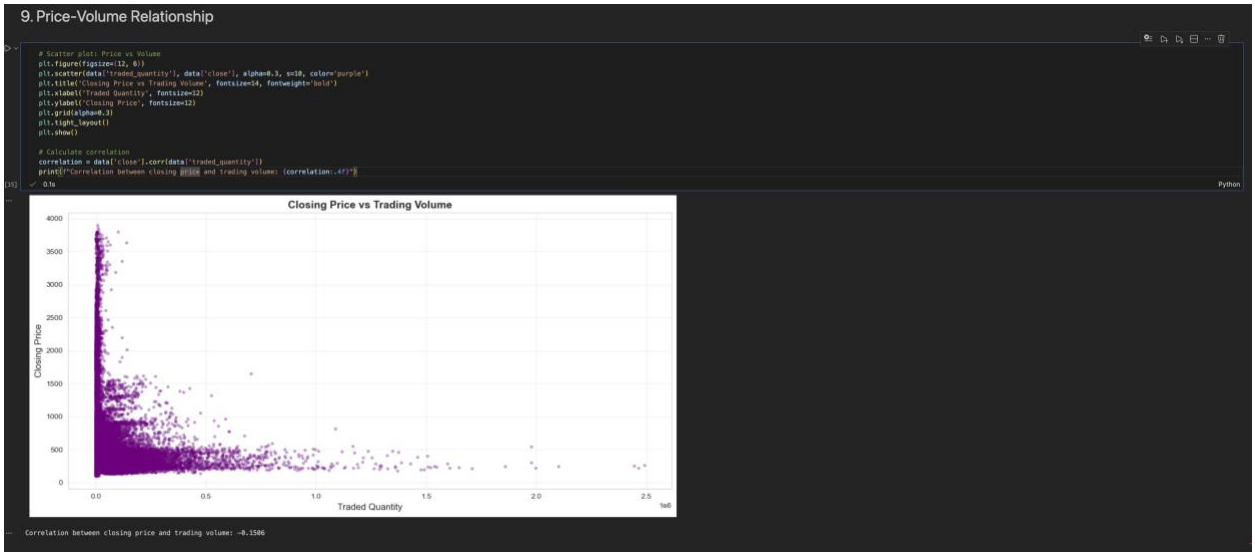


Figure 18: Trading Volume vs Closing Price

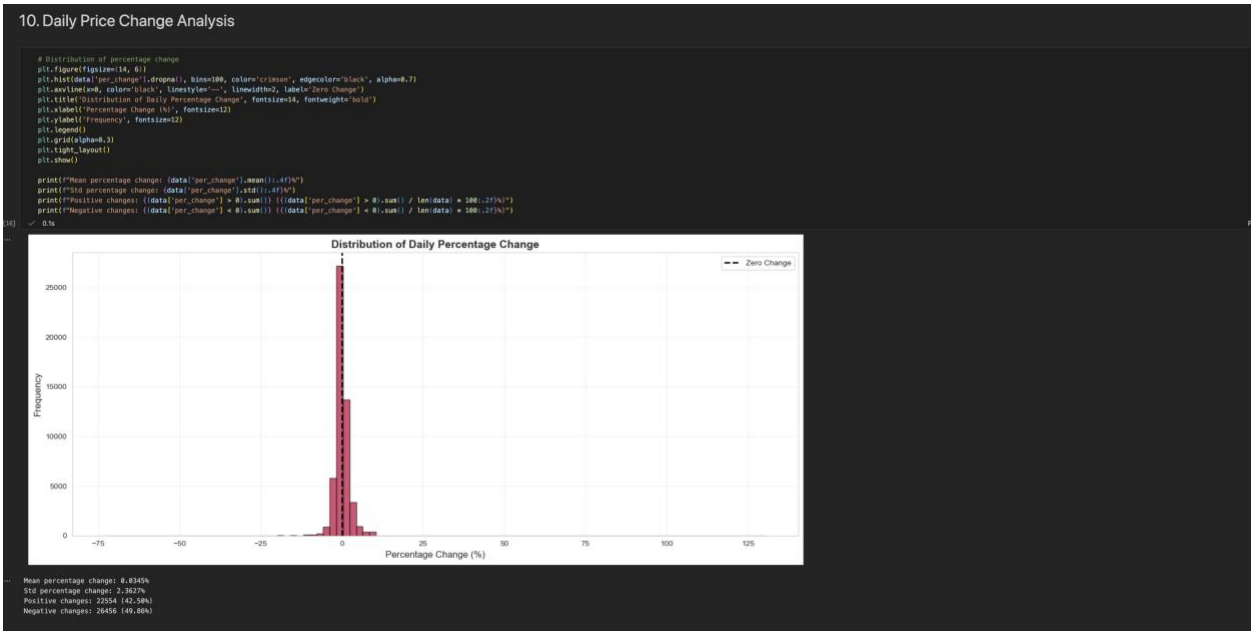


Figure 19: Daily Percentage Changes

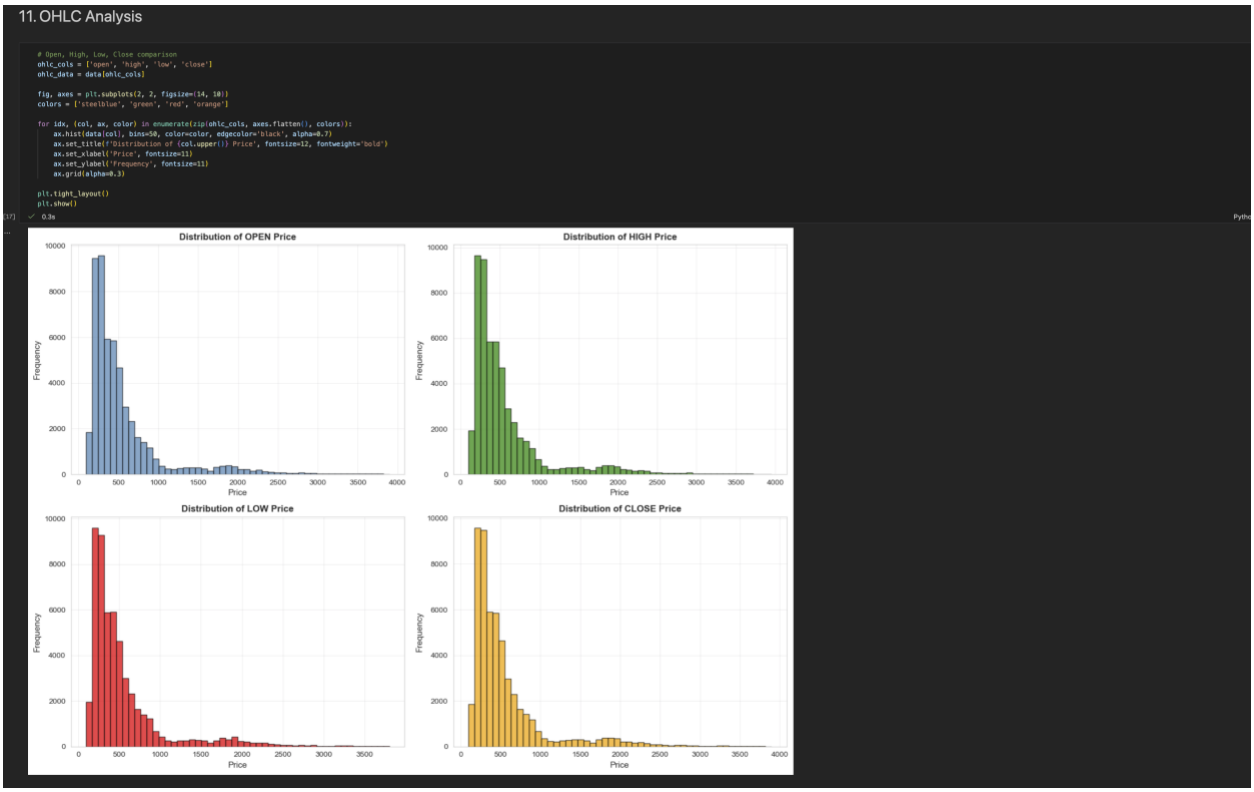


Figure 20: Open, Low, High, Close Prices Analysis

Figure 21: Temporal Coverage of Banks

4.4.1.4 Model Implementation

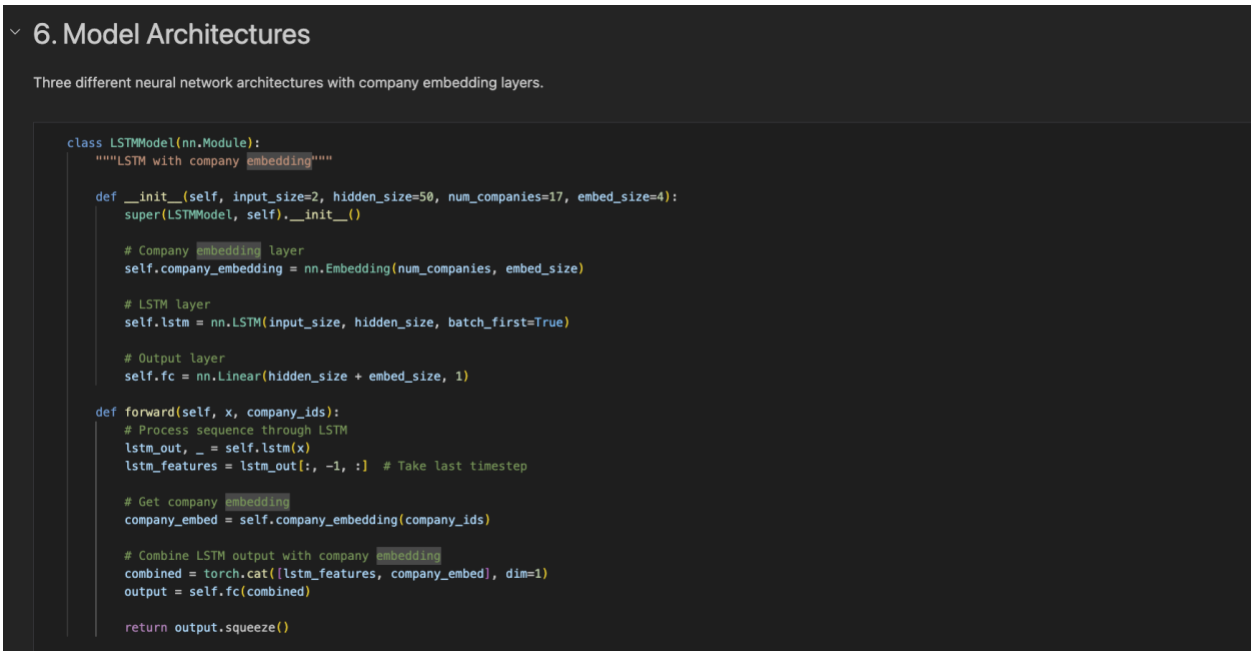


Figure 22: LSTM Model Architecture

```python
class CNNModel(nn.Module):
    """1D CNN with company embedding"""

    def __init__(self, input_size=2, num_companies=17, embed_size=4):
        super(CNNModel, self).__init__()

        # Company embedding layer
        self.company_embedding = nn.Embedding(num_companies, embed_size)

        # Convolutional layers
        self.conv1 = nn.Conv1d(input_size, 64, kernel_size=3, padding=1)
        self.pool1 = nn.MaxPool1d(2)
        self.conv2 = nn.Conv1d(64, 32, kernel_size=3, padding=1)
        self.pool2 = nn.MaxPool1d(2)

        # Fully connected layers
        self.flatten = nn.Flatten()
        self.fc = nn.Sequential(
            nn.Linear(480 + embed_size, 50),  # 32 * 15 = 480 after pooling
            nn.ReLU(),
            nn.Dropout(0.2),
            nn.Linear(50, 1)
        )
        self.relu = nn.ReLU()

    def forward(self, x, company_ids):
        # Transpose for Conv1d: (batch, features, sequence)
        x = x.permute(0, 2, 1)

        # Convolutional processing
        x = self.relu(self.conv1(x))
        x = self.pool1(x)
        x = self.relu(self.conv2(x))
        x = self.pool2(x)
        x = self.flatten(x)

        # Get company embedding
        company_embed = self.company_embedding(company_ids)

        # Combine CNN features with company embedding
        combined = torch.cat([x, company_embed], dim=1)
        output = self.fc(combined)

        return output.squeeze()
```

Figure 23: CNN Model Architecture

```python
class TransformerModel(nn.Module):
    """Transformer encoder with company embedding"""

    def __init__(self, input_size=2, d_model=32, nhead=4, num_layers=2,
                 num_companies=17, embed_size=4):
        super(TransformerModel, self).__init__()

        # Company embedding layer
        self.company_embedding = nn.Embedding(num_companies, embed_size)

        # Input projection
        self.input_proj = nn.Linear(input_size, d_model)

        # Transformer encoder
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=nhead,
            dim_feedforward=64,
            dropout=0.1,
            batch_first=True
        )
        self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=num_layers)

        # Output layers
        self.fc = nn.Sequential(
            nn.Linear(d_model + embed_size, 32),
            nn.ReLU(),
            nn.Linear(32, 1)
        )

    def forward(self, x, company_ids):
        # Project input to d_model dimensions
        x = self.input_proj(x)

        # Transformer processing
        x = self.transformer(x)
        x = x.mean(dim=1)  # Global average pooling

        # Get company embedding
        company_embed = self.company_embedding(company_ids)

        # Combine transformer output with company embedding
        combined = torch.cat([x, company_embed], dim=1)
        output = self.fc(combined)

        return output.squeeze()

print("Model architectures defined: LSTM, CNN, Transformer")
```

Figure 24: Transformer Model Architecture

4.4.1.5 Training Configuration

Common Settings Across All Models:

- **Loss Function:** MSELoss (Mean Squared Error for regression)
- **Optimizer:** Adam with learning_rate=0.001
- **Scheduler:** ReduceLROnPlateau (patience=5, factor=0.5)
- **Early Stopping:** Patience=15 epochs on validation loss
- **Max Epochs:** 50
- **Validation Split:** 90/10 temporal split of training data

```python
# Initialize models
models = {
    'LSTM': LSTMModel(input_size=2, num_companies=num_companies),
    'CNN': CNNModel(input_size=2, num_companies=num_companies),
    'Transformer': TransformerModel(input_size=2, num_companies=num_companies)
}

results = []
predictions_dict = {}
histories = {}

print("Training models...\n")
print("="*70)

for model_name, model in models.items():
    print(f"\nTraining {model_name}...")
    print("-"*70)

    # Train model
    model_trained, history = train_model(model, train_loader, epochs=50, lr=0.001)

    # Evaluate model
    pred, rmse, mae, r2, mape = evaluate_model(model_trained, test_loader, y_scaler, y_test)

    # Store results
    results.append({
        'Model': model_name,
        'RMSE': rmse,
        'MAE': mae,
        'R2': r2,
        'MAPE': mape
    })

    predictions_dict[model_name] = pred
    histories[model_name] = history

    print(f"\n{model_name} Results:")
    print(f"  RMSE: {rmse:.4f}")
    print(f"  MAE:  {mae:.4f}")
    print(f"  R2:   {r2:.4f}")
    print(f"  MAPE: {mape:.2f}%")

print("\n" + "="*70)
print("Training complete for all models")
```

```
Training models...

======================================================================

Training LSTM...
----------------------------------------------------------------------
Epoch 10/50 - Train Loss: 0.000036, Val Loss: 0.000107
Epoch 20/50 - Train Loss: 0.000038, Val Loss: 0.000099
Epoch 30/50 - Train Loss: 0.000028, Val Loss: 0.000110
Early stopping at epoch 35

LSTM Results:
  RMSE: 7.2964
  MAE:  4.3516
  R2:   0.9978
  MAPE: 1.31%
```

Figure 25: Model Training and evaluation (1)

```
Training CNN...
----------------------------------------------------------------------
Epoch 10/50 - Train Loss: 0.000327, Val Loss: 0.000687
Epoch 20/50 - Train Loss: 0.000295, Val Loss: 0.000107
Epoch 30/50 - Train Loss: 0.000277, Val Loss: 0.000129
Early stopping at epoch 35

CNN Results:
  RMSE: 8.0594
  MAE:  4.9142
  R2:   0.9973
  MAPE: 1.50%

Training Transformer...
----------------------------------------------------------------------
Epoch 10/50 - Train Loss: 0.000128, Val Loss: 0.001351
Early stopping at epoch 17

Transformer Results:
  RMSE: 19.4613
  MAE:  13.5412
  R2:   0.9840
  MAPE: 4.03%

======================================================================
Training complete for all models
```

Figure 26: Model Training and evaluation (2)

4.4.1.6 Evaluation

- Set model to evaluation mode (disable dropout/batch norm training)
- Generate predictions on test set (12,411 sequences)
- Inverse transform predictions using y_scaler to original price scale
- Calculate regression metrics comparing predictions to actual test prices

4.4.1.6.1 Metrics Calculated:

- RMSE (Root Mean Squared Error): √ (mean ((actual - predicted) ²))
- MAE (Mean Absolute Error): mean (|actual - predicted|)
- R² (R-squared): 1 - (SS_residual / SS_total)
- MAPE (Mean Absolute Percentage Error): mean (|actual - predicted| / actual) × 100

4.4.1.7 Final Test Results

Figure 27: Final Test Results

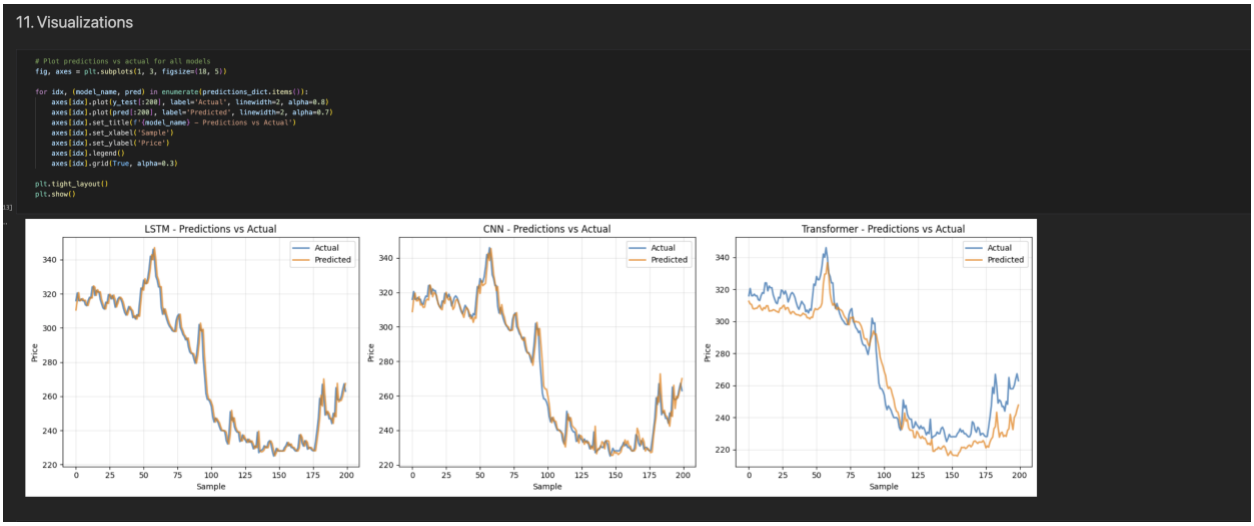


Figure 28: Final Tests Visualization

4.4.1.7.1 Key Findings:

- LSTM achieved best overall performance with lowest RMSE (7.30) and highest R^2 (0.9978)
- CNN performed competitively with slightly higher errors but faster training
- Transformer underperformed relative to LSTM/CNN, possibly due to:
- Limited sequence length (60 days) not leveraging attention advantages
- Smaller model capacity compared to optimal transformer configurations
- Need for more data or longer sequences to benefit from self-attention

4.4.2 Tools and Technologies

4.4.2.1 Programming:

- Python 3.9+

4.4.2.2 Primary Development Platform: Google Colab

Google Colab was selected as the primary development environment for several key reasons. It provides free access to NVIDIA Tesla T4 GPU with 16GB VRAM, eliminating the need for expensive local hardware. The platform comes with pre-installed deep learning frameworks (PyTorch, TensorFlow), saving setup time and preventing version conflicts. The browser-based Jupyter notebook interface enables interactive development with immediate code execution feedback. Integration with Google Drive allows easy data storage and retrieval.

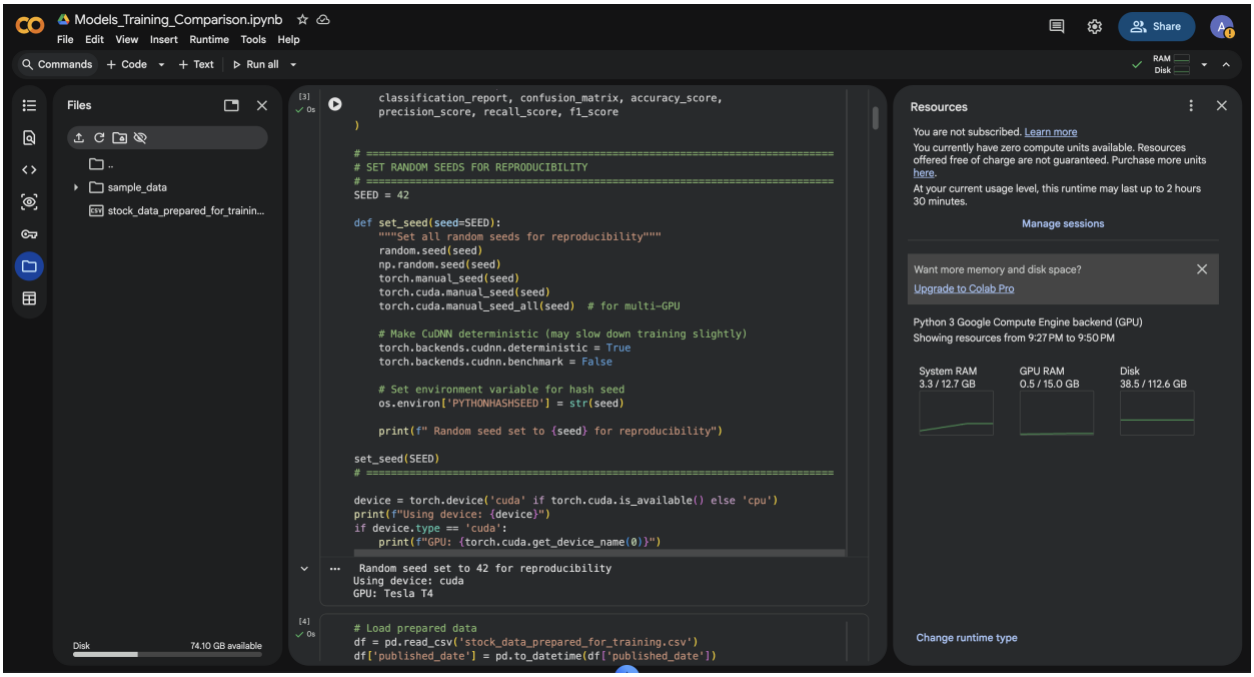


Figure 29: Using Cuda cores in colab

4.4.2.3 Libraries:

- Pandas (data handling)
- NumPy (numerical operations)
- Matplotlib/Seaborn (visualization)
- Pytorch (deep learning)
- Scikit-learn (preprocessing and metrics)

4.4.2.4 Environment:

- Jupyter Notebook
- Google Colab (GPU support)
- VS Code

4.4.2.5 Version Control:

- Git and GitHub
- Repository: https://github.com/AmiksKarki/Stock-Price-Prediction

4.4.2.6 Hardware:

GPU: NVIDIA Tesla T4

- Architecture: Turing
- CUDA Cores: 2,560
- Tensor Cores: 320
- Memory: 16 GB GDDR6
- Memory Bandwidth: 320 GB/s
- FP32 Performance: 8.1 TFLOPS
- Training acceleration: ~10-15x faster than CPU

System Memory: 12 GB RAM

- Sufficient for loading full dataset

- Handled batch processing without memory errors

Storage: ~100 GB in Colab environment

- Adequate for dataset (~13 MB) and model checkpoints

5 Conclusion

This paper considered the use of LSTM, CNN and Transformer model to stock market price prediction of closing prices of NEPSE. The common database was created in such a manner that the comparison of all the models would be equal. Formally speaking, LSTM engages the recognition of long-term trends, CNN gives consideration to short-term signals, and Transformers are high-performing using attention-based models.

Such new emerging markets as Nepal Stock Exchange will require the use of artificial intelligence forecasting to inform decision-making, support financial analysts and risk management. Although the presented work is merely a conceptual design of something rather than a real implementation outcome, the fact of the matter is that modern AI-based approaches could be employed to use in the process of solving real-life problems of finances prediction.

5.1 Evolution from Milestone 1

After getting a feedback of Mr. Binod Bhattarai, the scheme was changed to regression (actual price forecasting) rather than classification (price direction UP/DOWN). This became an advantage due to the following reasons:

- Better assessment and evaluation using regression measures.
- Stable and improved models.
- Streamlined company feature engineering.

5.2 Further Work

The path forward requires both deepening (improving current approach with better features, longer horizons, ensembles) and broadening (new techniques like RL, alternative data, hybrid models). The immediate focus should be:

- Using advanced models like Informer, LSTM-CNN hybrids, Temporal Fusion Transformers
- Test longer prediction horizons
- Implement ensemble methods

- Add datas such as sentiments, sector indices and macroeconomic indicators
- Trading Strategy Backtesting: Evaluate profitability with realistic transaction costs
- Study impact of data frequency (daily vs. hourly trading data)

6 References

Seetharaman, A. P. a. A., 2021. *Importance of Machine Learning in Making Investment Decision in Stock Market.* [Online] Available at: https://journals.sagepub.com/doi/10.1177/02560909211059992 [Accessed 12 December 2025].

Hapuarachchi, A. S., 2025. *THE IMPORTANCE OF MACHINE LEARNING,* s.l.: Research Gate.

Wenjie Lu, J. L. Y. L. A. S. J. W., 2020. *A CNN-LSTM-Based Model to Forecast Stock Prices.* [Online] Available at: https://onlinelibrary.wiley.com/doi/10.1155/2020/6622927 [Accessed 12 December 2025].

Nawa Raj Pokhrel, K. R. D. ,. R. R. ,. H. N. B. ,. R. K. K. ,. B. R. ,. W. E. H., 2022. *Predicting Nepse Index Price Using Deep Learning Models.* [Online] Available at: https://www.researchgate.net/publication/360921805_Predicting_Nepse_Index_Price_Using_Deep_Learning_Models [Accessed 12 December 2025].

Thomas Fischer, C. K., 2018. *Deep learning with long short-term memory networks for financial market predictions.* [Online] Available at: https://www.sciencedirect.com/science/article/abs/pii/S0377221717310652 [Accessed 12 December 2025].

T.O. Kehinde, W. A. K. S.-H. C., 2023. *Financial Market Forecasting using RNN, LSTM, BiLSTM, GRU and Transformer-Based Deep Learning Algorithms ,* Michigan: ieomsociety.

Omer Berat Sezera, M. O. E. D., 2017. *A Deep Neural-Network Based Stock Trading System Based on Evolutionary Optimized Technical Analysis Parameters.* [Online] Available at: https://www.researchgate.net/publication/320370508_A_Deep_Neural-Network_Based_Stock_Trading_System_Based_on_Evolutionary_Optimized_Technical_Analysis_Parameters

[Accessed 12 December 2025].

Ekanta Rai, S. T. S. U. J. K. L., 2025. *Stock Prediction Based on Transformer Model,* Kathmandu: NCCS Research Journal.

Sepp Hochreiter, J. S., 1997. *Long short-term memory,* s.l.: PubMed.

Avinash, 2021. *Hands-On Stock Price Time Series Forecasting using Deep Convolutional Networks.* [Online] Available at: https://www.analyticsvidhya.com/blog/2021/08/hands-on-stock-price-time-series-forecasting-using-deep-convolutional-networks/
[Accessed 12 December 2025].

Rogerio Pereira dos Santos, J. P. M.-C. V. R. Q. L., 2025. *Deep learning in time series forecasting with transformer models and RNNs.* [Online] Available at: https://peerj.com/articles/cs-3001/
[Accessed 12 december 2025].

Draxler, T. C. a. R. R., 2014. *Root mean square error (RMSE) or mean absolute error (MAE)? – Arguments against avoiding RMSE in the literature,* College Park: European Geosciences Union.

Kumar, A., 2023. *Mean Squared Error or R-Squared – Which one to use?.* [Online] Available at: https://vitalflux.com/mean-square-error-r-squared-which-one-to-use/
[Accessed 16 December 2025].

Liu, D., 2023. *Overview of Common Time Series Forecast Error Metrics.* [Online] Available at: https://www.dannidanliu.com/overview-of-common-time-series-forecast-error-metrics/
[Accessed 16 December 2025].